

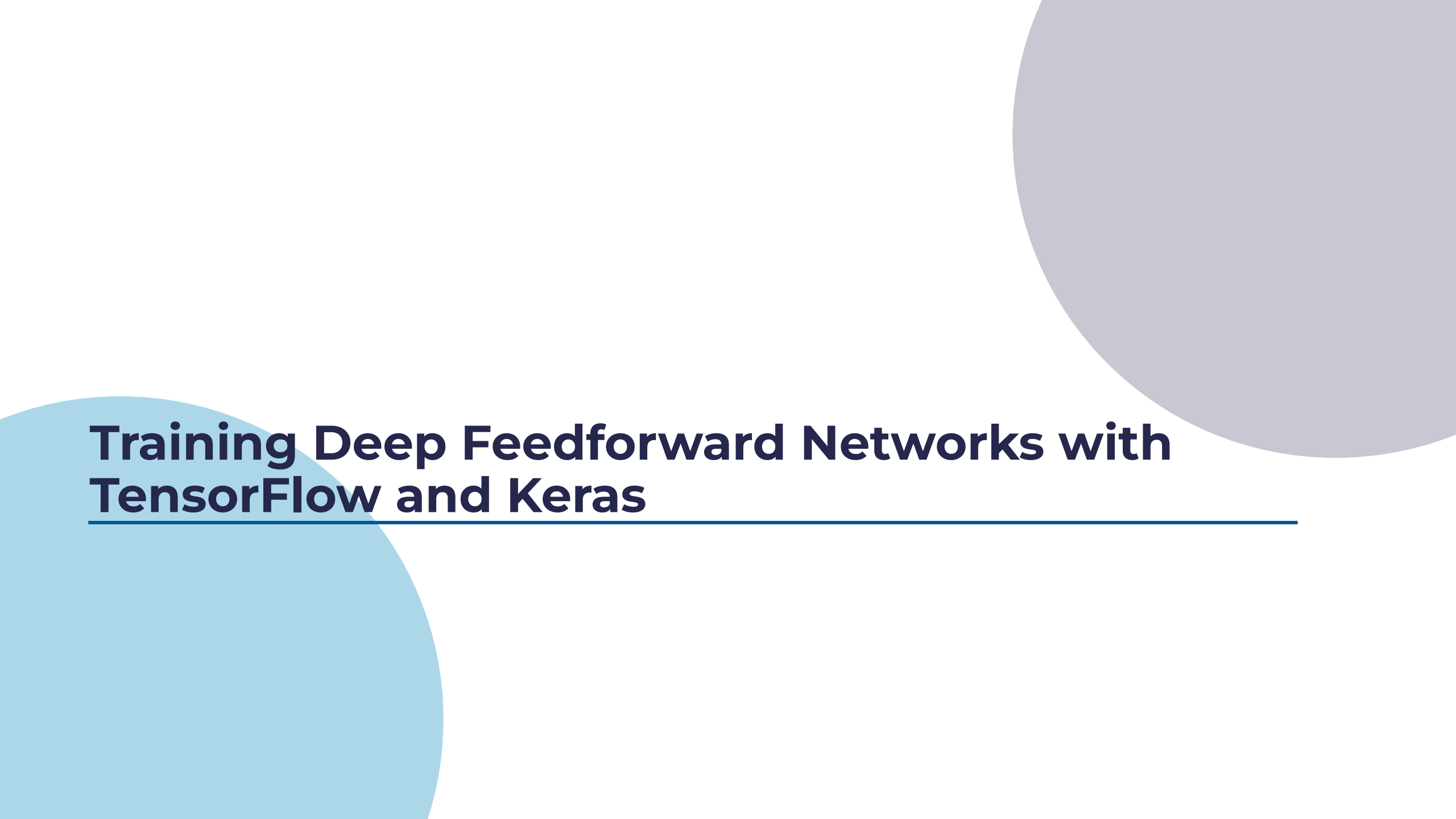


EMERITUS

Learn. From the world's best.

Multilayer Feedforward Neural Networks for Classification and Regression - II





Training Deep Feedforward Networks with TensorFlow and Keras

Today's Hands-On Goals

Practical Coding With Measurable Results

Learning Modes

Compare pattern, batch and mini-batch training in practice

Stopping Strategies

Compare pattern, batch and mini-batch training methods

Initialisation & Optimisation

Explore weight initialisation methods, multi-start strategies and optimiser choices

Network Architecture

Tune MLP depth and width while tracking convergence and accuracy

We'll apply async concepts with practical code and results

From Theory to Implementation

Asynchronous Materials Covered:

- Learning modes (SGD variants)
- Stopping criteria methodologies
- MLFFNN architecture
- Training considerations

Today's Implementation Focus:

- Run experiments to test concepts
- Compare training with metrics
- Apply best practices hands-on
- Build intuition for hyperparameters

Note: Focus on coding and results, not theory

Environment & Datasets

Required Libraries:

```
# Install dependencies  
pip install -U tensorflow scikit-learn matplotlib  
  
# Imports  
import tensorflow as tf  
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.datasets import make_classification  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler
```

Working Datasets:

- **Fashion-MNIST:** 28×28 grayscale clothing images, 10 classes
- **Synthetic:** 2D data for quick demos and visualisation

Experiment #1: Modes of Learning

Task: Compare Three Update Strategies

Pattern (SGD):

Update per sample

Set `batch_size=1`

Batch:

Update once per epoch

Set `batch_size=len(X_tr)`

Mini-batch:

Balance speed and noise

Set `batch_size=128` (typical value)

Experiment #1: Modes of Learning

Task: Compare Three Update Strategies

```
# Define a simple softmax classifier
model = tf.keras.Sequential([
    tf.keras.layers.Input((784,)),
    tf.keras.layers.Dense(10, activation='softmax')])

# Compile the model
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1), loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
hist = model.fit(x_tr, y_tr,
    epochs=10,
    batch_size=128, # Try 1 for SGD or len(x_tr) for batch gradient descent
    validation_split=0.1,
    verbose=0
)
```


Experiment #2: Stopping Criteria

Fixed Epochs vs. Early Stopping

Compare fixed training with validation-based stopping

```
# Early stopping callback
cb = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',    # Metric to monitor
        patience=3,            # Stop if no improvement after 3 epochs
        restore_best_weights=True
    )
]

# Train the model with early stopping
hist_es = model.fit(
    x_tr, y_tr,
    epochs=30,                # Maximum number of epochs
    batch_size=128,
    validation_split=0.1,
    callbacks=cb,
    verbose=0
)
```

What to Measure:

- Identify best epoch (minimum validation loss)
- Measure test accuracy
- Track training time
- Compare resource use

Experiment #3: Weight Initialisation

Compare Strategies For Training Stability And Performance

Different initialisation strategies can significantly impact training convergence and final model performance



Random Initialisation

Uniform values, no scaling
(*random_uniform*)



Glorot/Xavier

Scales by input/output, best for
sigmoid/tanh (*glorot_uniform*)



He Initialisation

Scales by input/output, best for
sigmoid/tanh (*glorot_uniform*)

Experiment #3: Weight Initialisation

Compare Strategies For Training Stability And Performance

```
# Function to build MLP with a given initialiser
def make_mlp(init):
    return tf.keras.Sequential([
        tf.keras.layers.Input((784,)),
        tf.keras.layers.Dense(
            256, activation='relu',
            kernel_initializer=init
        ),
        tf.keras.layers.Dense(10, activation='softmax')
    ])

# Test multiple initialisations with 3 random seeds each
for init in ['glorot_uniform', 'he_uniform', 'random_uniform']:
    tf.keras.utils.set_random_seed(42) # Ensure reproducibility
    m = make_mlp(init)
```

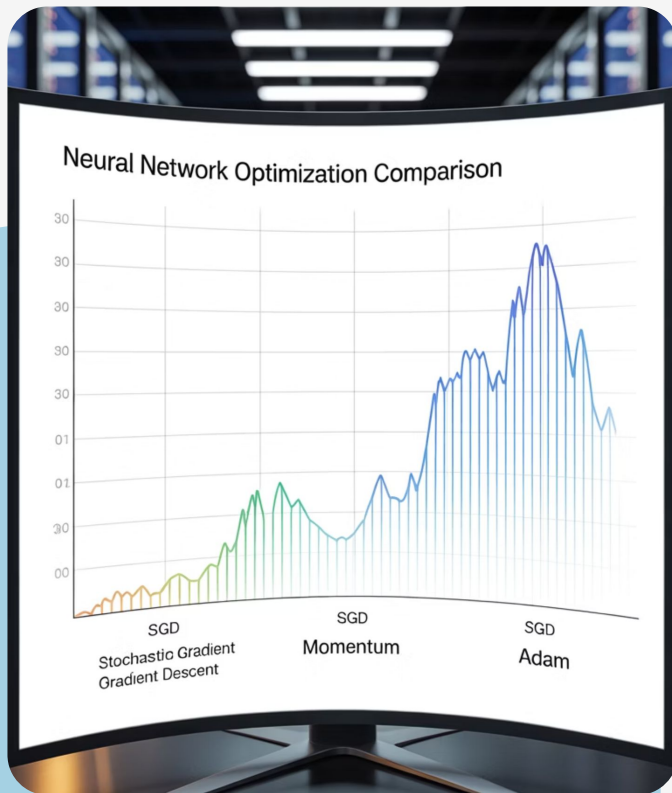
Experiment #3: Weight Initialization

Compare Strategies For Training Stability And Performance

```
m.compile(  
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.05, momentum=0.9),  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
m.fit(  
    x_tr, y_tr,  
    epochs=10,  
    batch_size=128,  
    validation_split=0.1,  
    verbose=0  
)  
print(init, m.evaluate(x_te, y_te, verbose=0)[1])
```

Experiment #4: Optimiser Bake-off

Compare optimisation algorithms with fixed architecture



1

SGD (Basic): Fixed learning rate
`tf.keras.optimisers.SGD(0.1)`

2

SGD + Momentum: Faster convergence with momentum
`tf.keras.optimisers.SGD(0.05, momentum=0.9)`

3

Adam: Adaptive learning rates per parameter
`tf.keras.optimisers.Adam(1e-3)`

Experiment #4: Optimiser Bake-off

Optional Enhancement: Learning Rate Schedule

```
# Reduce learning rate when validation loss plateaus  
sched = tf.keras.callbacks.ReduceLROnPlateau(  
    monitor='val_loss', # Metric to watch  
    factor=0.5,         # Halve the learning rate  
    patience=2          # Wait 2 epochs with no improvement  
)
```

Experiment #5: Architecture Capacity

Exploring Depth and Width Trade-offs

We'll vary depth (1–3 layers) and width (64–512 units) to test performance and overfitting

```
def build(depth=2, width=256, act='relu'):
    layers = [tf.keras.layers.Input((784,))]

    # Hidden layers
    for _ in range(depth):
        layers.append(tf.keras.layers.Dense(width, activation=act))

    # Output layer
    layers.append(tf.keras.layers.Dense(10, activation='softmax'))

    # Build and compile model
    m = tf.keras.Sequential(layers)
    m.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return m
```

Train and validation accuracy

Generalisation gap

Training time to converge

Parameters vs performance trade-off

Regularisation Techniques

Methods To Reduce Overfitting In Neural Networks



L2 Regularisation

Adds squared weight values to loss to penalise large weights

Implementation: `tf.keras.regularizers.l2(1e-4)`



Dropout

Randomly deactivates neurons during training to reduce co-adaptation

Implementation: `tf.keras.layers.Dropout(0.2)`

Regularisation Techniques

Methods To Reduce Overfitting In Neural Networks

```
# Define L2 regularizer
reg = tf.keras.regularizers.l2(1e-4)

# Model with L2 + Dropout
m = tf.keras.Sequential([
    tf.keras.layers.Input((784,)),
    tf.keras.layers.Dense(
        256, activation='relu',
        kernel_regularizer=reg
    ),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

Logistic Regression Baseline

Set a baseline first; for multiclass tasks, use softmax logistic regression

```
# Single-layer softmax classifier (logistic regression)
baseline = tf.keras.Sequential([
    tf.keras.layers.Input((784,)),
    tf.keras.layers.Dense(10, activation='softmax')
])

# Compile the model
baseline.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model
baseline.fit(x_train, y_train, epochs=10, batch_size=128, validation_split=0.1)
```

Sanity Check: If your MLP doesn't beat logistic regression, review implementation, hyperparameters, or task suitability.

Comparing with the baseline helps measure the gains from adding hidden layers and non-linearity to the model

Monitoring & Result Tracking

Essential Visualisation

```
def plot_history(history):  
    plt.figure(figsize=(12, 4))  
  
    # Loss plot  
    plt.subplot(1, 2, 1)  
    plt.plot(history.history['loss'], label='Training Loss')  
    plt.plot(history.history['val_loss'], label='Validation Loss')  
    plt.legend()  
    plt.title('Loss Curves')  
  
    # Accuracy plot  
    plt.subplot(1, 2, 2)  
    plt.plot(history.history['accuracy'], label='Training Accuracy')  
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
    plt.legend()  
    plt.title('Accuracy Curves')  
    plt.tight_layout()  
    plt.show()  
  
history = baseline.fit(x_tr, y_tr, epochs=10, batch_size=128, validation_split=0.1)  
plot_history(history)
```

Monitoring & Result Tracking

Experiment Tracking

Create a structured results table:

Model	Train Acc	Val Acc	Epochs	Time (s)
Baseline LR	0.855	0.850	10	5.2
MLP-1×256	0.875	0.872	8	7.8
MLP-2×256	0.891	0.887	12	13.5

Optional: Use TensorBoard for advanced visualisation

```
tb_callback = tf.keras.callbacks.TensorBoard(log_dir='runs/w25',  
histogram_freq=1)
```

Hands-on Deliverables & Troubleshooting

Required Deliverables:

1

Learning Modes Report: Pattern vs batch vs mini-batch (time/accuracy trade-offs)

2

Stopping Criteria Analysis: Best epoch & test score (early stopping vs fixed epochs)

3

Weight Init & Optimiser Report: Results table of different strategies

4

Architecture Sweep: Depth/width impact chart + confusion matrix

5

Key Findings: 3–5 points on what improved performance most and why

Hands-on Deliverables & Troubleshooting

Troubleshooting Common Issues

Loss Flat or NaN

- Lower learning rate
- Normalise inputs
- Use He/Xavier init

Overfitting

- Add dropout or L2
- Apply early stopping
- Reduce model size

Underfitting

- Increase layers/units
- Train longer
- Use Momentum/Adam

Unstable Training

- Apply gradient clipping
- Use LR scheduling
- Ensure proper normalisation

Q&A Session



Open floor for questions and clarifications.